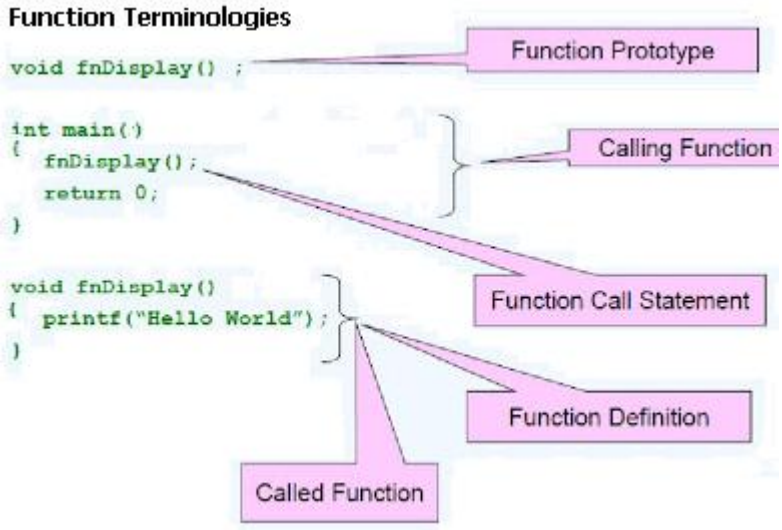
MODULE III(COND.): FUNCTIONS

FUNCTION IN C

• A function is a block of code to perform a specific task.
• Every C program has at least one function main( ). Without main() function, there is technically no C program.
• Program to print a sentence using function is shown below.



TYPES OF C FUNCTIONS

• There are 2 types of functions in C programming:
→ Library function
→ User defined function

Library Function

• Library functions are the in-built function in C compiler.
For example:
→ main() //The execution of every C program starts from this main() function
→ printf() //prinf() is used for displaying output in C
→ scanf() //scanf() is used for taking input in C

User Defined Function

• C allows programmer to define their own function according to their requirement. These types of functions are known as user-defined functions.
• Suppose, a programmer wants to find factorial of a number and check whether it is prime or not in same program. Then, he can create two separate user-defined functions in that program:
→ one for finding factorial and
→ other for checking whether it is prime or not

ADVANTAGES OF USER DEFINED FUNCTIONS

1) User defined functions helps to decompose the large program into small segments which makes programmer easy to understand, maintain and debug.
2) If repeated code occurs in a program. Function can be used to include those codes and execute when needed by calling that function.
3) Programmer working on large project can divide the workload by making different functions.

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

FUNCTIONS AND PROGRAM STRUCTURE

• Basic structure of C program with function is shown below:

```
#include <stdio.h>
void function_name();        //function declaration
int main()
{
.....
.....
function_name();             // function call
.....
.....
}

void function_name()         //function definition
{
.....
.....
}
```

• As mentioned earlier, every C program begins from main() and program starts executing the codes inside main() function.

• When the control of program reaches to function_name() inside main() function, the control of program jumps to void function_name() and executes the codes inside it.

• When all the codes inside that user-defined function are executed, control of the program jumps to the statement just after function_name() from where it is called.

Function Declaration

• Every function in C program should be declared before they are used.

• Function declaration gives compiler information about

→ function name

→ type of arguments to be passed and

→ return type

• The syntax is shown below:

```
return_type function_name(type(1) argument(1),....,type(n) argument(n));
```

Function Call

• Control of the program cannot be transferred to user-defined function unless it is called invoked.

• The syntax is shown below:

```
function_name(argument(1),....argument(n));
```

Function Definition

• Function definition contains programming codes to perform specific task.

• The syntax is shown below:

```
return_type function_name(type(1) argument(1),..,type(n) argument(n))
{
        //body of function
}
```

• Example: Program to Print a sentence using function.

```
#include<stdio.h>
void display();                    //function declaration

void main()
{
    display();                     //function call
}
void display()                     //function definition
{   printf("C Programming");
    return;
}
```

Output:

```
C Programming
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

ACTUAL AND FORMAL ARGUMENTS

• Argument (or parameter) refers to data that is passed to function (function definition) while calling function.

• Arguments listed in function calling statements are referred to as actual arguments.
These actual values are passed to a function to compute a value or to perform a task.

• The arguments used in the function declaration are referred as formal arguments.
They are simply formal variables that accept or receive the values supplied by the calling function.

• The number of actual and formal arguments and their data types should be same.

• Example: Program to add two integers. Make a function add integers and display sum in main() function.

```
#include <stdio.h>
int add(int a, int b);
int main()
{
    int a,b,sum;
    printf("Enters two number to add \n");
    scanf("%d %d", &a,&b);          //actual arguments
    sum=add(a,b);
    printf("\n sum=%d", sum);
    return 0;
}
int add(int a,int b)                //formal arguments
{
    int sum;
    sum=a+b;
    return sum;
}
```

Output:

```
Enters two number to add
2 3
sum=5
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

ARGUMENT PASSING – CALL BY VALUE

• In this type, value of actual arguments are passed to the formal arguments and the operation is done on the formal arguments.
• Any changes made in the formal arguments does not effect the actual arguments because formal arguments are photocopy of actual arguments.
• Changes made in the formal arguments are local to the block of called-function.
• Once control returns back to the calling-function the changes made vanish.
• Example: Program to send values using call-by-value method.

```
#include <stdio.h>
void main()
{       int x,y,
        printf("enter values of x & y : ");
        scanf("%d %d ", &x, &y);
        printf("\n old values x=%d y =%d", x, y);
        change(x,y) ;
        printf("\n new values x=%d y =%d", x, y);
}
void change(int a, int b)
{
        k=a;
        a=b;
        b=k;
        return;
}
```

Output:

```
enter values of x & y : 2 3
old values x=2 y =3
new values x=2 y =3
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

CLASSIFICATION OF USER-DEFINED FUNCTIONS

1. Function with no arguments and no return value
2. Function with no arguments and return value
3. Function with arguments but no return value
4. Function with arguments and return value

Example: Program to illustrate function with no arguments and no return value.

```
#include <stdio.h>
void add();
void main()
{
    add();
}
void add()
{
    int a, b, sum;
    printf("Enters two number to add : ");
    scanf("%d %d", &a, &b);
    sum=a+b;
    printf("\n sum=%d", sum);
    return;
}
```

Output:

```
Enters two number to add : 2 3
sum=5
```

Example: Program to illustrate function with no arguments and return value.

```
#include <stdio.h>
int add();
void main()
{
    int sum;
    sum=add();
    printf("\n sum=%d", sum);

}
int add()
{
    int a, b, sum;
    printf("Enters two number to add \n");
    scanf("%d %d", &a, &b);
    sum=a+b;
    return sum;
}
```

Output:

```
Enters two number to add
2 3
sum=5
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

Example: Program to illustrate function with arguments but no return value

```
#include <stdio.h>
void add(int a, int b);
void main()
{
    int a, b, sum;
    printf("Enters two number to add \n");
    scanf("%d %d", &a, &b);
    add(a, b);
}
void add(int a, int b)
{
    sum= a+ b;
    printf("\n sum=%d", sum);
    return;
}
```

Output:

```
Enters two number to add
2 3
sum=5
```

Example: Program to illustrate function with arguments and return value

```
#include <stdio.h>
int add(int a, int b);
int main()
{
    int a, b, sum;
    printf("Enters two number to add \n");
    scanf("%d %d", &a, &b);
    sum=add(a,b);
    printf("\n sum=%d", sum);
    return 0;
}

int add(int a, int b)
{
    int sum;
    sum=a+b;
    return sum;                //return statement of function
}
```

Output:

```
Enters two number to add
2 3
sum=5
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

RECURSION

- A function that calls itself is known as recursive function.
- Example: Program to find sum of first n natural numbers using recursion.

```
#include <stdio.h>

int add(int n)
{
   if(n==0)
        return n;
   else
       return n+add(n-1);    /*self call  to function add() */
}

void main()
{
   int num, sum;
   printf("Enter a positive integer: ");
   scanf("%d",&num);
   sum=add(num);
   printf("sum=%d", sum);
}
```

Output:

```
Enter a positive integer: 5
15
```

Explanation:

- Here, add() function is invoked from the same function.
- If n is not equal to 0 then, the function calls itself passing argument 1 less than the previous argument it was called with.
- Suppose, n is 5 initially. Then, during next function calls, 4 is passed to function and the value of argument decreases by 1 in each recursive call.
- When, n becomes equal to 0, the value of n is returned which is the sum numbers from 5 to 1.
- Observe the following for better visualization of recursion in this example:

```
add(5)
=5+ add(4)
=5+4+ add(3)
=5+4+3+ add(2)
=5+4+3+2+ add(1)
=5+4+3+2+1+ add(0)
=5+4+3+2+1+0
=5+4+3+2+1
=5+4+3+3
=5+4+6
=5+10
=15
```

- Every recursive function must be provided with a way to end the recursion. In this example when, *n* is equal to 0, there is no recursive call and recursion ends.

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

STORAGE CLASS

- Every variable in C program has two properties: type and storage class.
- Type refers to the data type of variable whether it is character or integer or floating-point value etc.
- And storage class determines how long it stays in existence.
- There are 4 types of storage class:
  1) automatic(or local)
  2) external(or global)
  3) static
  4) register

AUTOMATIC VARIABLE

- Variables declared inside the function body are automatic by default.
- These variables are also known as local variables as they are local to the function and doesn't have meaning outside that function.
- For example:
  int a,b; or auto int a,b;
- Since, variable inside a function is automatic by default, keyword auto are rarely used.

EXTERNAL VARIABLES

- External variable can be accessed by any function. They are also known as global variables.
- Variables declared outside every function are external variables.
- In case of large program, containing more than one file, if the global variable is declared in file 1 and that variable is used in file 2 then, compiler will show error.
- To solve this problem, keyword extern is used in file 2 to indicate that, the variable specified is global variable and declared in another file.
- Example: Program to demonstrate working local and global variables.

```
#include <stdio.h>
int z;                          // z is a global variable

void display()
{
      printf("\n sum=%d", z);
}

void main()
{                        // a and b are local variables
    auto int a, b;     //or int a, b;
    printf("Enters two number to add: ");
    scanf("%d %d", &a, &b);
    z=a+b;
    display();
}
```

Output:

```
Enters two number to add: 2 3
sum=5
```

REGISTER VARIABLES

- Register variables are similar to automatic variable and exists inside that particular function only.
- For example:
  register int z;
  where z is register variable
- If the compiler encounters register variable, it tries to store variable in microprocessor's register rather than memory.
- Value stored in register are much faster to access than that of memory.
- In case of larger program, variables that are used in loops and function parameters are declared register variables.

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

Example: Program to illustrate use of register variable.

```
#include <stdio.h>
void main()
{
    int a, b;
    register int z;
    printf("Enters two number to add \n");
    scanf("%d %d", &a, &b);
    z=a+b;
    printf("\n sum=%d", z);
}
```

Output:

```
Enters two number to add
2 3
sum=5
```

STATIC VARIABLE

- The value of static variable persists until the end of the program.
- For example:

  static int c;

  where c is a static variable
- Example: Program to demonstrate working of static variable.

```
#include <stdio.h>
void Check()
{
   static int c=0;
   printf("%d \t",c);
   c=c+5;
}

int main()
{
  Check();
  Check();
  Check();
}
```

Output:

```
0     5     10
```

Explanation:

- During first function call, it will display 0.
- Then, during second function call, variable c will not be initialized to 0 again, as it is static variable.
- So, 5 is displayed in second function call and 10 in third call.

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

MODULE IV: STRUCTURES

INTRODUCTION

• Suppose, you want to store the information about person about his name, citizenship number and salary.

• You can create these information separately but, better approach will be collection of these information under single name because all these information are related to person.

• For this purpose, structure can be used.

STRUCTURES

• Structure is a collection of elements of different data type.

• The syntax is shown below:

```
struct structure_name
{
   data_type member1;
   data_type member2;
   data_type member3;
};
```

• The variables that are used to store the data are called members of the structure.

• We can create the structure for a person as shown below:

```
struct person
{
   char name[50];
   int cit_no;
   float salary;
};
```

• This declaration above creates the derived data type struct person.

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

Structure Variable Declaration

• A structure can be declared using 2 methods:

1) Tagged Structure

• When a structure is defined, it creates a user-defined type but, no storage is allocated. For the above structure of person, variable can be declared as:

```
struct person
{
   char name[50];
   int cit_no;
   float salary;
};
Inside main function:
struct person p1, p2;
```

• Here the above statement declares that the variables p1 and p2 are variables of type struct person.

• Once the structure variable is declared, the compiler allocates memory for the structure variables.

• The size of the memory allocated is the sum of size of individual members.

2) Type Defined Structure

• Another way of creating sturcture variable using the keyword typedef is:

```
typedef struct person
{
   char name[50];
   int cit_no;
   float salary;
}EMP;
Inside main function:
EMP p1 ,p2 ;
```

• Here the above statement declares that the variables p1 and p2 are variables of type EMP(which is of type as struct person)..

Structure Variable Initialization

• For ex:

```
struct person
{
   char name[50];
   int cit_no;
   float salary;
};
struct person p1={"ram", 24, 23000};
```

Accessing Members of a Structure

• Member operator(.) can be used for accessing members of a structure.

• Any member of a structure can be accessed as:

structure_variable_name.member_name

• By specifying p1.name, we can access the name 'ram'.

By specifying p1.salary, we can access the value '23000'.

• The various values can be accessed and printed as shown below:

```
printf("%s", p1.name);
printf("%f", p1.salary);
```

• We can read the members of a structure as shown below:

```
gets(p1.name); or scanf("%s", p1.name);
scanf("%f",&p1.salary);.
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

STRUCTURES WITHIN STRUCTURES

• For ex:

```
struct dob        //dob = date of birth
{
        int day;
        int month;
        int year;
};

struct person
{
        char name[50];
        int cit_no;
        float salary;
        struct dob d1;
};

Inside main:
struct person p1 ,p2 ;
```

• Suppose you want to access month for p1 structure-variable then, structure-member p1.d1.month is used.

SELF REFERENTIAL STRUCTURES

• A self-referential structure is one in which one or more of its components is a pointer to itself.

• These require dynamic storage management functions (malloc & free) to explicitly obtain and release memory.

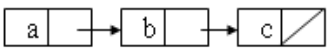
```
typedef struct
{
    char data;
    struct list *link;    //list is a pointer to a list structure
 }list;
```

• Consider three structures and values assigned to their respective fields:

```
list item1,item2,item3;
item1.data='a';
item2.data='b';
item3.data='c';
item1.link=item2.link=item3.link=NULL;
```

• We can attach these structures together as follows:

```
item1.link=&item2;
tem2.link=&item3;
```



For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

STRUCTURES AND FUNCTIONS

• In C, structure can be passed to functions by 2 methods:
1) Passing by value (passing actual value as argument)
2) Passing by reference (passing address of an argument)

Passing Structure by Value

• A structure-variable can be passed to the function as an argument as normal variable.

• If structure is passed by value, change made in structure-variable in function-definition does not reflect in original structure variable in calling-function.

• Example: Write a C program to create a structure student, containing name and roll. Ask user the name and roll of a student in main function. Pass this structure to a function and display the information in that function.

```
#include<stdio.h>
struct student
{
   char name[50];
   int roll;
};

void Display(struct student stu)
{
   printf("Your Name: %s  \n",stu.name);
   printf("Your Roll: %d",stu.roll);
}

void main()
{
   struct student s1;
   printf("Enter student's name: ");
   scanf("%s", &s1.name);
   printf("Enter roll number: ");
   scanf("%d", &s1.roll);
   Display(s1);          //passing structure variable s1 as argument
}
```

Output:

```
Enter student's name: rama
Enter roll number: 149
Your Name: rama
Your Roll: 149
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

Passing Structure by Reference

• The address location of structure-variable is passed to function while passing it by reference.

• If structure is passed by reference, change made in structure-variable in function-definition reflects in original structure variable in the calling-function.

• Write a C program to add two complex numbers entered by user. To solve this program, make a structure. Pass two structure variable (containing distance in real and imaginary part) to add function by reference and display the result in main function without returning it.

```
#include <stdio.h>
struct comp
{
   int real;
   float img;
};

void Add(struct comp c1,struct comp c2, struct comp *c3)
{
    // Adding complex numbers c1 and d2 and storing it in c3
    c3->real=c1.real+c2.real;
    c3->img=c1.img+c2.img;
}

void main()
{
   struct comp c1, c2, c3;
   printf("First complex number \n");
   printf("Enter real part: ");
   scanf("%d",&c1.real);
   printf("Enter imaginary part: ");
   scanf("%f",&c1.img);
   printf("Second complex number \n");
   printf("Enter real part: ");
   scanf("%d",&c2.real);
   printf("Enter imaginary part: ");
   scanf("%f",&c2.img);
   Add(c1, c2, &c3);

   //passing structure variables c1 and c2 by value whereas passing
   // structure variable c3 by reference
   printf("Sum of 2 complex numbers = %d + i %d",c3.real, c3.img);
}
```

Output:

```
First complex number
Enter real part: 3
Enter imaginary part: 4
Second complex number
Enter real part: 2
Enter imaginary part: 5
Sum of 2 complex numbers = 5 + i 9
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/